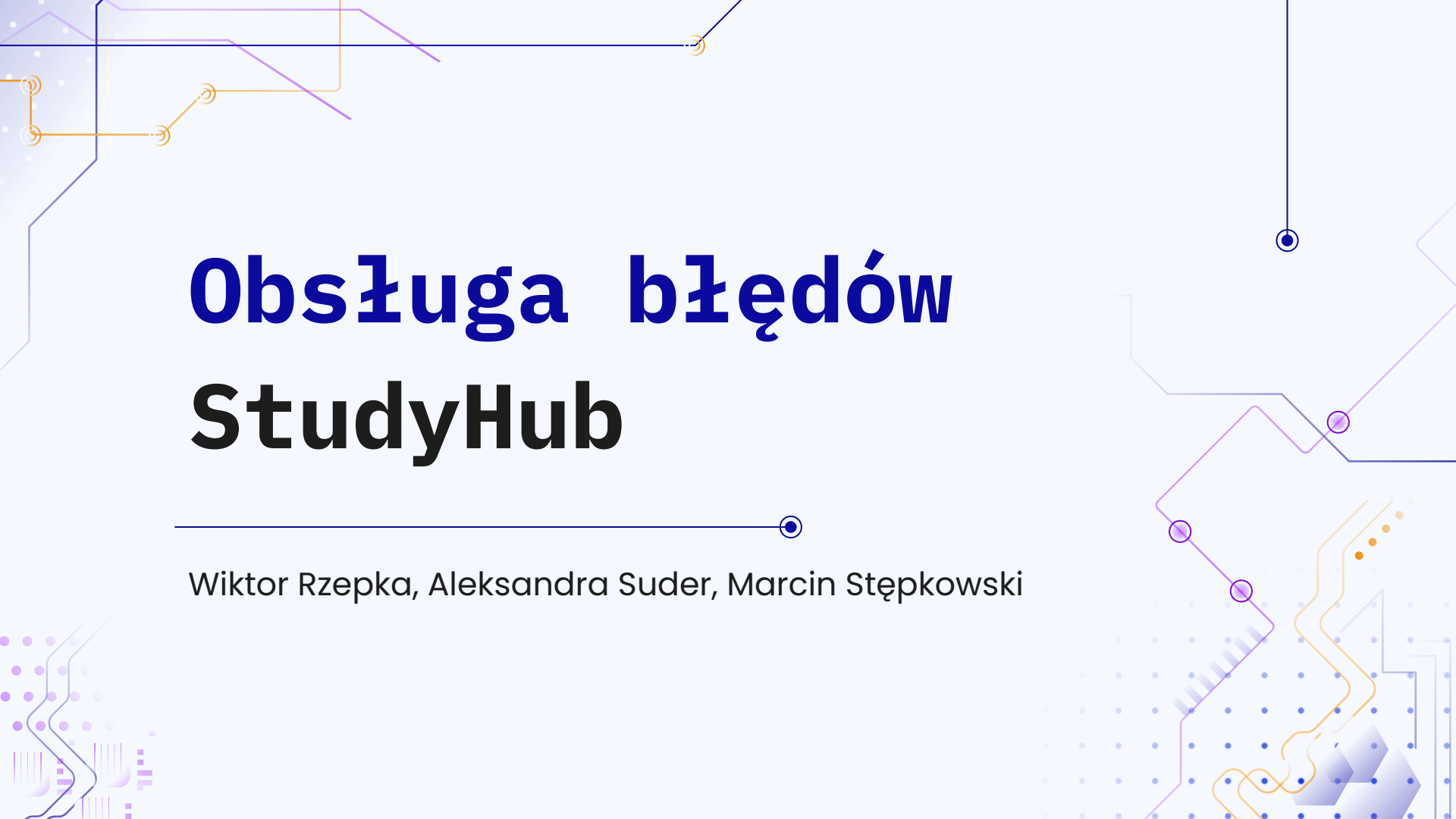




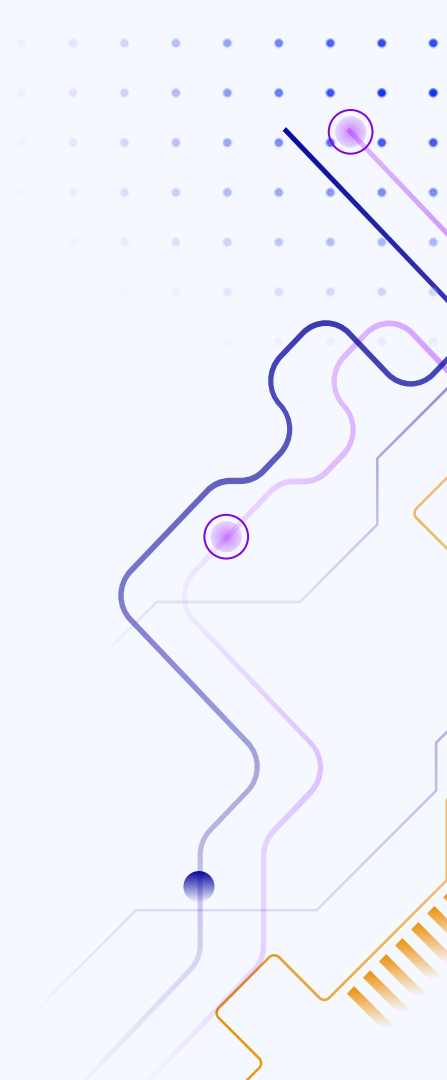
Obsługa błędów StudyHub

Wiktor Rzepka, Aleksandra Suder, Marcin Stępkowski



01

Logowanie



Poziomy logów

Używamy standardowych poziomów logging Pythona:

- DEBUG – dostępny po ustawieniu `debug=True` w `configure_logging`.
- INFO – poziom bazowy dla normalnych zdarzeń (start, operacje masowe).
- WARNING – używany w walidacji / sytuacjach nietypowych (np. w innych modułach).
- ERROR – błędy operacji (wyjątki, błędy sieciowe).
- CRITICAL – poważne błędy (np. testowy `demo_critical`).

Logger znajduje się w data/logger.py. Zapisuje on logi do data/logs/app.log w formie JSON'ów z podstawowymi polami (session_id, user, wersja aplikacji).

Przy starcie aplikacji wywołujemy funkcję configure_logging i install_global_excepthook, więc każda nieobsłużona akcja/UI wpada do logów zamiast wysypać aplikację.

Gdy w projekcji znajduje się AZURE_LOG_CONNECTION_STRING, automatycznie podpinamy AzureLogHandler i logi przesyłane są do Azure Application Insights, gdzie znajduje się cały monitoring oraz alerty

```
def configure_logging(debug: bool = False) -> logging.Logger:
    """Konfiguruje root logger: JSON, rotacja plików, opcjonalne alerty."""
    _ensure_env_loaded()

    log_dir = DEFAULT_LOG_DIR
    log_dir.mkdir(parents=True, exist_ok=True)

    root = logging.getLogger()
    root.handlers.clear()
    root.setLevel(logging.DEBUG if debug else logging.INFO)

    file_handler = RotatingFileHandler(DEFAULT_LOG_FILE, maxBytes=MAX_BYTES, backupCount=BACKUP_COUNT, encoding="utf-8")
    file_handler.lock = threading.RLock()
    file_handler.setFormatter(JsonFormatter())
    file_handler.addFilter(_context_filter)
    root.addHandler(file_handler)

    # Wysyłka do Azure Application Insights (jeśli podano connection string)
    connection_string = os.getenv("AZURE_LOG_CONNECTION_STRING", "")
    if connection_string:
        azure_handler = AzureLogHandler(connection_string=connection_string)
        azure_handler.lock = threading.RLock()
        azure_handler.addFilter(_context_filter)
        root.addHandler(azure_handler)
        root.info("azure_handler_attached", extra={"event": "azure_handler_attached"})

    # Alerty przez webhook (jeśli ustawiono LOG_ALERT_WEBHOOK)
    webhook_url = os.getenv("LOG_ALERT_WEBHOOK")
    alert_handler = AlertHandler(webhook_url)
    alert_handler.lock = threading.RLock()
    alert_handler.addFilter(_context_filter)
    root.addHandler(alert_handler)

    root.info("logger_initialized", extra={"event": "logger_initialized"})
    return root
```

02

Wyjątki (Try / Catch)

SafeApplication w main.py owija każde zdarzenie UI w try/except, dzięki czemu nieobsłużone wyjątki nie crashują aplikacji, tylko są bezpiecznie obsługiwane.

install_global_excepthook w main.py zbiera niezłapane exceptions globalnie i przejmuję kontrolę zamiast defaultowego crasha Pythona.

W logowaniu użytkownika try/except w login_overlay.py i auth_service.py, które zamieniają błędy na komunikaty dla użytkownika.

```
class SafeApplication(QApplication):
    """QApplication z globalnym try/except dla zdarzeń UI (graceful failure + log)."""

    def __init__(self, argv, logger: logging.Logger):
        super().__init__(argv)
        self.logger = logger

    def notify(self, receiver, event):
        try:
            return super().notify(receiver, event)
        except Exception as exc:
            self.logger.error(
                "Unhandled UI exception",
                exc_info=exc,
                extra={"event": "ui_exception"},
            )
            QMessageBox.critical(None, "Błąd", f"Wystąpił nieoczekiwany błąd:\n{exc}")
            return False
```

Globalny excepthook

Wszystko, co nie zostało złapane, jest logowane jako CRITICAL z eventem "uncaught_exception".

```
def install_global_excepthook(logger: logging.Logger) -> None:
    """Przechwytuje nieobsłużone wyjątki, loguje jako CRITICAL i próbuje zakończyć elegancko."""

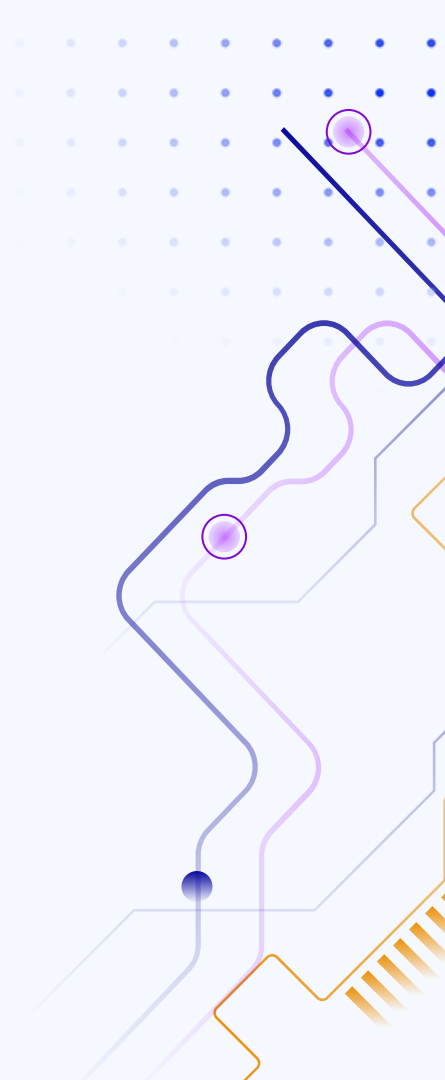
    def _hook(exc_type, exc_value, exc_tb):
        logger.critical(
            "Uncaught exception",
            exc_info=(exc_type, exc_value, exc_tb),
            extra={"event": "uncaught_exception"},
        )
        # Pozwalamy defaultowemu hookowi wyświetlić trace jeśli jest w dev
        sys.__excepthook__(exc_type, exc_value, exc_tb)

    sys.excepthook = _hook
```



03

Monitoring



Logi data/logger.py lokalnie zapisuje w formie JSON do data/logs/app.log z informacjami session_id, user, wersja.

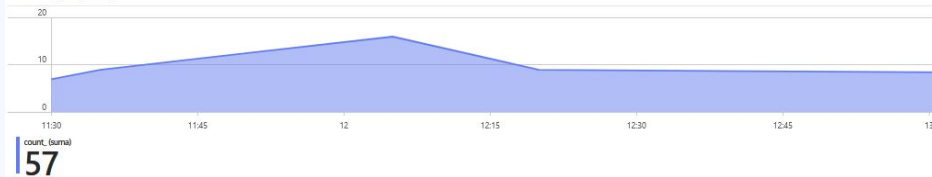
Monitoring działa jeśli w .env/azure.json jest AZURE_LOG_CONNECTION_STRING. Wtedy automatycznie podpinany jest AzureLogHandler i logi lecą do Application Insights.

W Application Insights mamy skoroszyt z wykresami (liczba logów, błędy severity ≥ 3 [error / critical], ostatnie wpisy) oraz alert e-mail na severityLevel >= 3 w ciągu 5 minut.

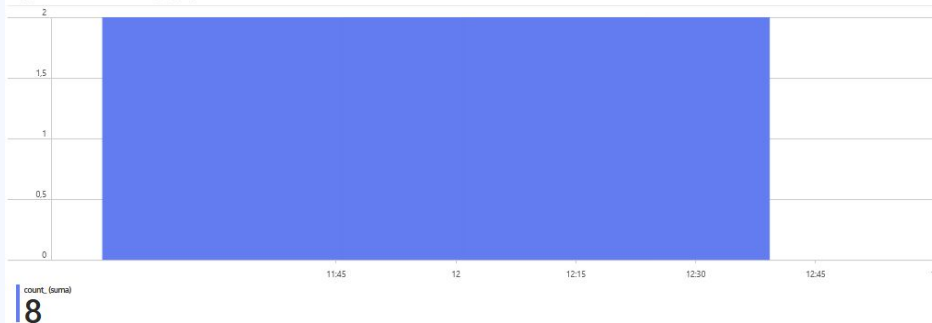
StudyHUB

Skoroszyt przedstawiający logi z aplikacji

Liczba logów (5 min)



Błędy (ERROR/CRITICAL) – agregacja 5 min



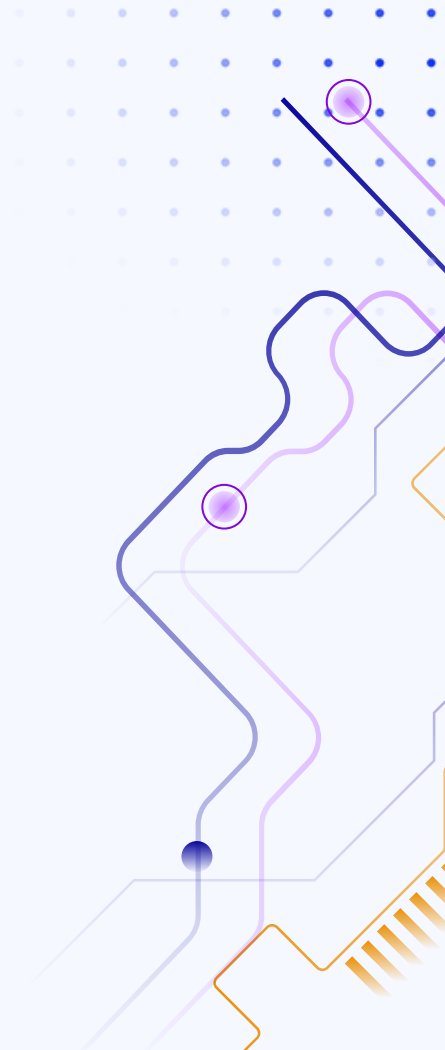
Ostatnie wpisy

timestamp	↑↓	severityLevel↑↓	message	↑↓	event	↑↓	user	↑↓
10.12.2025, 15:00:55,165		2	Microsoft config load failed from C:\Users\wikto\OneDriv...					
10.12.2025, 15:00:53,846		2	Microsoft config load failed from C:\Users\wikto\OneDriv...					
10.12.2025, 15:00:53,732		2	Microsoft config load failed from C:\Users\wikto\OneDriv...					
10.12.2025, 15:00:53,637		1	HTTP Request: GET https://rintbredmgrjxdajic.supabase...					
10.12.2025, 15:00:53,234		1	HTTP Request: GET https://rintbredmgrjxdajic.supabase...					
10.12.2025, 15:00:47,550		4	Demo critical triggered for alert test					
10.12.2025, 15:00:47,497		4	Demo critical triggered for alert test					
10.12.2025, 15:00:47,494		1	logger_initialized					
10.12.2025, 15:00:47,493		1	azure_handler_attached					
10.12.2025, 14:57:55,782		2	Microsoft config load failed from C:\Users\wikto\OneDriv...					
10.12.2025, 14:57:53,425		2	Microsoft config load failed from C:\Users\wikto\OneDriv...					



04

Alerty



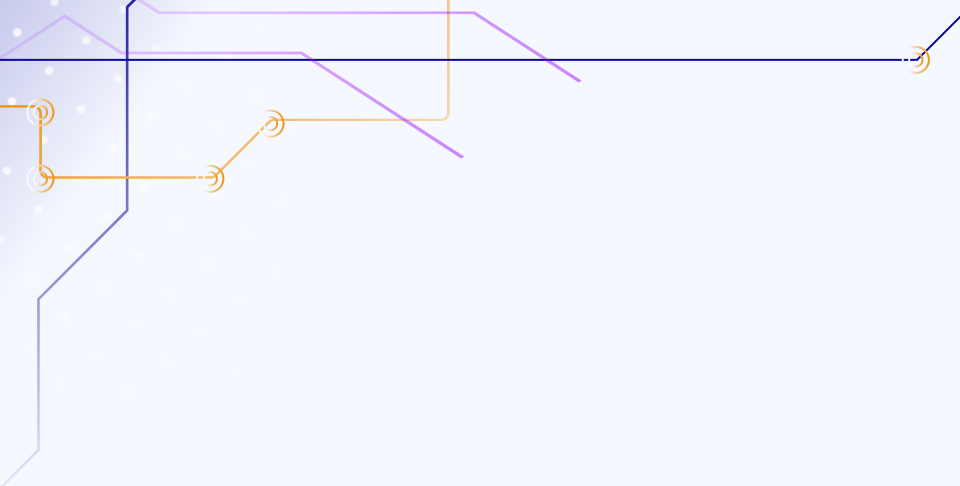


Łączna liczba alertów		Krytyczne	Błąd	Ostrzeżenie	Informacyjny	Pełne	Brak grupowania	
3		3	0	0	0	0		
Nazwa ↑↓		Ważność ↑↓		Dotknięty zasób ↑↓		Warunek alertu ↑↓		Odpowiedź użytkownika ↑↓
☐ ai-studyhub-errors		0 - Krytyczne		ai-studyhub		Wyzwolono		Nowy
☐ ai-studyhub-errors		0 - Krytyczne		ai-studyhub		Wyzwolono		Nowy
☐ ai-studyhub-errors		0 - Krytyczne		ai-studyhub		Wyzwolono		Nowy

Reguła w Azure Application Insights to:
traces | where severityLevel >= 3 | where timestamp > ago(5m), próg > 0,
ocena co 5 min, priorytet Krytyczny.

Gdy reguła zaszła to jako alert wysyłany jest e-mail z informacją o wystąpieniu errora / critical errora.

Dla testowania działania alertów ustawiamy zmienną lokalną. Po uruchomieniu programu w AI pojawia się wpis demo_critical (severity 4) a po 1–3 min przychodzi e-mail z informacją o wystąpieniu errora.

A decorative graphic in the top-left corner featuring a network of thin, intersecting lines in blue, purple, and orange. Some lines terminate in small circular nodes, resembling a circuit board or a data network diagram.

KONIEC

Wiktor Rzepka, Aleksandra Suder, Marcin Stępkowski

A decorative graphic in the bottom-right corner featuring a grid of small blue dots. Overlaid on this grid are stylized, wavy lines in orange and blue, along with some geometric shapes like triangles and squares, creating a modern, technological aesthetic.